1 Member 1 - Web Performance Profiling Engine

Build a custom browser performance profiler similar to the main functions available in the Chrome DevTools Performance panel.

Primary responsibility

- Own the complete live website performance-testing module from URL validation to final visual results.
- Collect reliable browser runtime, page-loading, main-thread, memory and network measurements.
- Provide normalized performance data to the carbon and recommendation modules through internal APIs.

Development tasks

- **Live scan engine:** Validate a URL, launch Puppeteer or Chrome DevTools Protocol, manage timeouts, errors and scan status.
- **Test profiles:** Support desktop and mobile viewports, CPU throttling and common network conditions.
- **Core Web Vitals:** Measure FCP, LCP, CLS, TBT, Speed Index, Time to Interactive and server response time.
- **Performance timeline:** Capture navigation start, DNS, connection, request, response, DOM events, paint events and final load.
- **Main-thread profiling:** Measure JavaScript execution, rendering, layout, painting, idle time and long tasks.
- **Memory and DOM:** Record initial, peak and final memory usage, DOM node count and possible memory growth.
- **Network profiling:** Collect request count, file type, status, transfer size, loading time, domain and blocking behavior.
- **Scoring:** Design a transparent performance score and severity classification for detected issues.
- **API integration:** Expose structured scan results for storage, charts, carbon calculations and AI recommendations.

Dashboard pages and UI components

- Live Performance Test page with URL, device and network controls.
- Performance Overview with score cards and Core Web Vitals.
- Interactive Performance Timeline.
- Network Requests table with filters.
- CPU and Main Thread Analysis.
- Memory and DOM Analysis.
- Performance issue details and technical evidence.

Individual research contribution

- Investigate how browser runtime metrics can be combined to create a custom performance profiler for modern web applications.
- Compare the custom profiler results with Chrome DevTools or Lighthouse and document differences, limits and accuracy.

- Define measurement rules that are repeatable across multiple test runs.

Testing and evaluation

- Run repeated tests on static, image-heavy, JavaScript-heavy and poorly optimized websites.
- Measure result consistency, scan duration, failure handling and metric deviation from a reference tool.
- Unit-test URL validation, timing calculations and score generation.

Expected deliverables

- Working browser performance profiling service and internal API.
- All performance dashboard pages and charts.
- Test dataset, comparison results and individual evaluation report.
- Technical documentation covering metrics, algorithms and known limitations.

Completion criterion: A user can run a live website test and receive repeatable performance, network, CPU, memory and timeline results that can be compared with a recognized browser tool.